

Python Temelleri

İlginç ve Faydalı Notlar

37 başlık · 6 kategori

İçindekiler

1. Bilgisayarlar ve Veri Tipleri
2. String (Metin) İşlemleri
3. Koleksiyon Yapıları: List, Set, Dict, Tuple
4. Fonksiyonlar ve Lambda
5. Nesne Yönelimli Programlama (OOP)
6. Dosya İşlemleri ve Hata Yönetimi

1. Bilgisayarlar ve Veri Tipleri

Bilgisayarlar verileri nasıl ayırt eder?

Bilgisayarlar yalnızca 0 ve 1 ile çalışır. Aynı değer farklı tiplerle farklı anlam taşır:

```
65 → tam sayı olabilir  
65 → ASCII tablosunda 'A' harfini temsil eder
```

- Veri tipi, bir değerın sayı mı harf mi olduğunu belirler. Yorum bağlama göre değişir.

int vs float — bellek farkı

```
myInt = 11  
myFloat = 1.33333  
# myInt sayısal olarak büyük, ama myFloat bellekte daha fazla yer kaplar
```

- int: sabit boyut, hızlı işlem. float: ondalık hassasiyet, daha fazla bellek.

Bölme işlemi her zaman float döndürür

```
40 / 4 # Sonuç: 10.0 (int değil!)
```

- Diğer birçok dilden farklı olarak Python'da / operatörü her zaman float verir.

✓ Tam sayı bölmesi için // operatörünü kullan: 40 // 4 → 10

Floating Point Hassasiyet Problemi

```
4.4 - 2  
# Sonuç: 2.4000000000000004
```

- Bu bir hata değil. Ondalık sayılar binary sistemde tam temsil edilemez.

- Kesin ondalık işlemler için decimal modülü kullanılabilir.

İsmlendirme Stilleri

```
# camelCase → ilk kelime küçük, diğerleri büyük harfle başlar  
myVariableName  
  
# snake_case → kelimeler alt çizgiyle ayrılır (Python standardı)  
my_variable_name
```

✓ Python'da PEP8 standardına göre snake_case tercih edilir.

2. String (Metin) İşlemleri

String tırnak tuzağı

```
# Hata: tek tırnak içinde apostrof  
'Nemo's place' # SyntaxError  
  
# Doğru yollar:  
"Nemo's place" # çift tırnak  
'Nemo\'s place' # escape karakteri ile
```

Stringler dizi gibi çalışır (indeksleme)

```
myString = "Hello Python"
myString[-1] # Son karakter: 'n'
myString[len(myString)-1] # Aynı sonuç
```

- Negatif indeks sondan saymaya başlar. -1 son eleman, -2 sondan bir önce.

Slicing — Dilimleme

```
barcode[start:stop:step]
# start → dahil, stop → hariç, step → adım

barcode[1:5] # 1'den 4'e kadar
barcode[::2] # çift indeksler
barcode[::-1] # ters çevir (Python trick!)
```

- ✓ `[::-1]` stringi tersine çevirmenin en kısa Python yoludur.

Immutability — Stringler değiştirilemez

```
myString[0] = "a" # TypeError: 'str' object does not support item assignment
```

- Stringler immutable'dır. Değişiklik için yeni string oluşturmak gerekir.

```
myList[0] = 100 # Listeler mutable'dır, değiştirilebilir
```

String birleştirme ve çoğaltma

```
"A" + "B" # Sonuç: "AB" (concatenation)
fullname * 5 # Stringi 5 kez yan yana yazar
```

Kaçış Karakterleri (Escape Characters)

```
\n → yeni satır
\t → tab boşluğu
\\ → literal backslash
\' → literal tek tırnak
```

String metodlarını keşfetme

```
name. # nokta koyup TAB → tüm metodlar listelenir
help(name.count) # metodun ne yaptığını açıklar
name.capitalize # fonksiyon nesnesi (çalışmaz!)
name.capitalize() # parantez ekle → çalışır
```

- Parantez olmadan yazılan fonksiyon çalışmaz, sadece referans döner.

3. Koleksiyon Yapıları

List — Değiştirilebilir dizi

```
list1 = [10, 20, 30]
list1[0] # 10 – indeksle erişim
list1 * 2 # [10,20,30,10,20,30] – çoğaltma
# list1 / 5 # Bölme yapılamaz!
```

Boş Set tuzağı

```
emptySet = {} # Bu dict oluşturur!
type(emptySet) #

emptySet = set() # Gerçek boş set
```

■ Python'da {} boş set değil, boş sözlük (dict) oluşturur!

Set — Sırasız, indeksiz koleksiyon

```
mySet[0] # TypeError: 'set' object is not subscriptable
# Setlerde sıra yoktur, indeksle erişilemez
```

in operatörü — Üyelik kontrolü

```
10 in [10, 20, 30] # Liste
10 in {10, 20, 30} # Set
10 in (10, 20, 30) # Tuple

# Dictionary'de değer arama:
10 in my_dict.values()
```

Dictionary — Güvenli erişim

```
# Anahtar yoksa KeyError:
fitness_dict["apple"] # hata olabilir

# Güvenli yol – varsayılan değerle:
fitness_dict.get("apple", 0) # yoksa 0 döndürür
```

✓ get() metodu uygulamanın çökmesini önler. Üretim kodunda tercih edilmeli.

Tuple Unpacking

```
my_list = [(1, "ali"), (2, "veli")]

for (x, y) in my_list:
    print(x, y) # 1 ali / 2 veli
```

– Tuple unpacking, çiftli veya çoklu listelerde döngü kurmayı kolaylaştırır.

input() her zaman string döndürür

```
x = input("Sayı gir: ")
type(x) #

# Sayısal işlem için dönüşüm:
x = int(input("Sayı gir: "))
```

pass — Yer Tutucu

```
def fonksiyon():
    pass # Sonradan doldurulacak

if koşul:
    pass # Boş bırakmak yerine pass yaz
```

4. Fonksiyonlar ve Lambda

****kwargs — İsimli parametreler**

```
def kwargs_example(**kwargs):  
    print(kwargs)  
  
kwargs_example(apple=100, banana=150, melon=200)  
# Çıktı: {'apple': 100, 'banana': 150, 'melon': 200}
```

- **kwargs fonksiyona gönderilen tüm isimli argümanları dictionary olarak toplar.

map() — Toplu uygulama

```
def divideNumber(n):  
    return n / 2  
  
myList = [10, 20, 30]  
result = list(map(divideNumber, myList))  
# Sonuç: [5.0, 10.0, 15.0]
```

- map() bir fonksiyonu koleksiyonun tüm elemanlarına uygular. Sonuç map nesnesidir, list() ile dönüştür.

Lambda — Anonim fonksiyon

```
multiplyLambda = lambda num: num * 3  
multiplyLambda(5) # 15  
  
# Genel sözdizimi:  
lambda parametre: ifade
```

- Lambda, def kullanmadan tek satırda fonksiyon tanımlar. Bir kerelik kullanım için idealdir.

Lambda + map() birlikte

```
numList = [4, 8, 12, 16]  
result = list(map(lambda num: num / 4, numList))  
# Sonuç: [1.0, 2.0, 3.0, 4.0]
```

- ✓ Kısa dönüşümler için lambda + map kombinasyonu çok kullanışlıdır.

5. Nesne Yönelimli Programlama (OOP)

Kalıtım (Inheritance)

```
class Musician:  
    def play(self):  
        print("Playing...")  
  
class MusicianPlus(Musician): # Musician'dan kalıtım  
    def record(self):  
        print("Recording...")
```

- Alt sınıf, üst sınıfın tüm metod ve özelliklerini miras alır.

Private Attribute — Kapsülleme

```
class Product:
    def __init__(self, price):
        self.__price = price # private: sadece sınıf içinden
        self.name = "item" # public: her yerden erişilir
```

- Çift alt çizgi (__) ile tanımlanan özellikler dışarıdan erişilemez.

Soyutlama (Abstraction)

```
from abc import ABC, abstractmethod

class Car(ABC):
    @abstractmethod
    def maxSpeed(self):
        pass # Alt sınıflar bu metodu uygulamak ZORUNDA
```

- ABC (Abstract Base Class) ile soyut sınıf oluşturulur. @abstractmethod dekoratörü alt sınıfları zorlar.

__str__ — Nesneyi yazdırma

```
class Food:
    def __init__(self, name, calories):
        self.name = name
        self.calories = calories

    def __str__(self):
        return f"{self.name}: {self.calories} calories"

f = Food("Apple", 52)
print(f) # Apple: 52 calories
```

- __str__ metodu, nesne print() ile yazdırıldığında görüntülenecek metni belirler.

__getitem__ — Köşeli parantez erişimi

```
class MyCollection:
    def __getitem__(self, key):
        return self.name

obj = MyCollection()
obj[0] # __getitem__ tetiklenir
```

- Bu özel metod, nesneyi liste gibi köşeli parantezle erişilebilir kılar.

6. Dosya İşlemleri ve Hata Yönetimi

Dosya açma modları

```
# Üzerine yaz (write)
with open("dosya.txt", mode="w") as f:
```

```
f.write("yeni içerik")

# Sonuna ekle (append)
with open("dosya.txt", mode="a") as f:
    f.write("eklenen satır")
```

– mode='w' önceki veriyi siler. mode='a' önceki veriyi korur, sonuna ekler.

✓ with open(...) as f: yapısı dosyayı otomatik kapatır.

finally bloğu

```
try:
    x = int(input("Sayı: "))
except ValueError:
    print("Geçersiz değer")
finally:
    print("Bu HERZAMAN çalışır") # hata olsa da olmasa da
```

– finally: kaynakları serbest bırakmak (dosya, bağlantı kapatma) için kullanılır.

Python'da öğrendiğin en önemli şeylerden biri: basit görünen özelliklerin çok güçlü olması, okunabilir kodun önemszenmesi ve küçük syntax detaylarının büyük kolaylık sağlamasıdır.